Class 6: R functions

Mikaela Joya (PID: A17295169)

Table of contents

Background

All functions in R have at least 3 things:

- a name (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function),
- the *body* (the lines of R code that do the work of the function).

A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either **"required"** or **"optional"**. The later have fall-back default values that will be used if the user does not specify them.

Q1: Write your first R function: add()

Q1a.Your first version of the function should add two input numbers together. For example, add(4,7) should return 11.

```
# Define a function called add that takes two inputs (x and y)
# y has a default value of 0
add <- function(x, y=0) {
# Return the sum of x and y
  x + y
}
```

Can we use our new function:

```
# Example usage
add(4,7)
```

```
[1] 11
```

Q1b. For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)).

```
# Redefine add() so it can either have (1) 2 separate numbers or (2) a single vector of numbe
add <- function(x, y=0) {
# Use sum() to add all values in x and y together
# This works whether x is a single value/vector
  sum(x,y)
}
```

```
# Example usage
add(4, 7)
```

```
[1] 11
```

```
add( c(4,7,10) )
```

```
[1] 21
```

Q1c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum()

```r
# Final version of add() that can take any number of inputs
# The ... argument lets the user pass in multiple values
add <- function(...) {
# sum() adds all values through ...
  sum(...)
}
```

```r
# Example usage
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

```r
# Define a function add() that takes up to 3 inputs
# y and z have default values of 0 (optional)
add <- function(x, y=0, z=0) {
# The default is that R returns the result of the last line in the function
# Return the sum of x, y, and z
  sum(x,y,z)
}

# Test the function with 2 inputs (z defaults to 0)
add(10,100)
```

```
[1] 110
```

```r
# Define add() again, using return()
add <- function(x, y = 0, z=0) {
# return() sends this value back as the function output
# After return (), the function stops
  return(sum(x, y, z))
# Line will not run because return() already exists
  cat("Is it break time yet")
}
```

Q2.Write a generate_dna() function

A useful function here is the "base R" `sample()` function:

```
# Randomly select 3 values from 1-5
# By default, values have no repeats
sample(1:5, size=3)
```

```
[1] 4 1 3
```

```
# Randomly select 60 values from 1-5
sample(1:5, size=60,
# replace = TRUE allows the same number to be selected multiple times
       replace = TRUE)
```

```
 [1] 2 1 1 5 2 2 4 1 1 5 5 2 3 4 4 4 4 3 5 5 1 5 1 1 3 1 1 5 5 3 2 1 2 4 1 3 4 3
[39] 2 5 2 1 1 3 3 2 4 1 4 5 2 5 5 1 1 2 4 2 4 3
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "G", and "T"...

```
# Randomly generate a sequence of 60 nucleotides
# Function samples from the vector of DNA bases
# replace = TRUE ensures nucleotides can repeat
sample(x=c("A","G","C","T"), size= 60, replace = TRUE)
```

```
 [1] "A" "G" "T" "G" "G" "C" "C" "C" "G" "A" "C" "T" "G" "T" "T" "C" "G" "G" "T"
[20] "C" "A" "T" "A" "C" "A" "T" "G" "T" "A" "C" "T" "A" "T" "C" "T" "A" "G" "A"
[39] "C" "G" "C" "C" "A" "G" "G" "A" "T" "A" "G" "C" "A" "G" "T" "T" "C" "C" "C"
[58] "G" "T" "A"
```

> **Q2**: Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. **Q2a**. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C".

```
# Define a function to generate a random DNA sequence
generate_dna <- function(len=10) {

# Randomly sample 'len' nucleotides from A, G, C, and T
# replace = TRUE allows nucleotides to repeat
  sample(x=c("A","G","C","T"), size= len, replace = TRUE)
}
```

```
# Call function to generate DNA sequence with length 100
# Output will be a vector of 100 random nucleotides
generate_dna(len=100)
```

```
  [1] "T" "G" "T" "C" "G" "G" "G" "G" "G" "A" "T" "G" "A" "C" "G" "A" "T" "T"
 [19] "T" "A" "T" "T" "C" "T" "G" "A" "A" "C" "A" "A" "G" "T" "T" "A" "T" "C"
 [37] "C" "G" "G" "A" "A" "C" "G" "G" "G" "A" "A" "T" "C" "G" "G" "A" "C" "C"
 [55] "C" "G" "T" "A" "C" "C" "T" "C" "T" "T" "A" "G" "T" "G" "T" "C" "G" "A"
 [73] "T" "T" "G" "A" "G" "A" "T" "C" "C" "T" "T" "A" "C" "A" "A" "G" "A" "G"
 [91] "G" "C" "C" "T" "T" "G" "T" "T" "T" "T"
```

Q2b. Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG".

```
# Redefine generate_dna() to optionally return a single DNA string
generate_dna <- function(len= 10, single.element=FALSE) {

#Generate a random DNA sequence as a vector of nucleotides
  ans <- sample(x=c("A","G","C","T"), size=len, replace = TRUE)

#If single.element is TRUE, convert the nucleotide vector into 1 string
  if(single.element) {
    return(
# Collapse the nucleotide vector into 1 DNA string
    paste( ans, collapse = ""))
  }
# Otherwise, return the sequence as a vector of individual nucleotides
  return(ans)
}
```

Functions that could be useful here are `paste()`, `if()`, `cat()` and `return()`

```
# Return a vector of nucleotides (default)
generate_dna()
```

```
 [1] "C" "C" "T" "C" "T" "T" "A" "T" "A" "C"
```

```
# Return a single continuous DNA string
generate_dna(single.element = TRUE)
```

```
[1] "ACTCTCCCTC"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: len9 CGAAGGCTG Be sure to include clear comments that explain each step of your function.

```
# Example of using cat() to print text
# "\n" is a newline character moving "there" to the next line
cat("hello \nthere")
```

```
hello
there
```

```
generate_dna <- function(len= 10, single.element=TRUE) {

# Randomly sample nucleotides A, G, C, and T
# Number sampled is determined by the specified length

  ans <- sample(x=c("A","G","C","T"), size=len, replace = TRUE)

# Generate random nucleotides
# Collapse vector into 1 continious DNA string

 seq <- paste( ans, collapse = "")

# Format as FASTA with an ID line
  cat(paste(">len", len, "\n",seq, sep="") )

}
```

```
# Expected behavior
generate_dna(9)
```

```
>len9
CGTTTGAAT
```

Q3. Write a `generate_protein()` function

> **Q3.** Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG". Your function should:Use the single-letter code for all 20 standard amino acids and no other letters (see earlier list at the beginning of this handout)and include clear comments that explain each step of your function.The one-letter code for the 20 standard amino acids is: "A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T", "W", "Y", "V"

```
# Define a function to generate a random protein sequence
# Input 'len' is the desired length of the protein (6)
generate_protein <- function(len= 6) {
# Store each letter codes for the 20 standard amino acids
  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T
# Randomly sample amino acids to make a protein to the desired length
  ans <- sample(x=aa, size=len, replace= TRUE)
# Collapse the amino acid vector into one continuous peptide
  paste(ans, collapse = "")
}
```

```
# Call the function to generate a random protein sequence with length 6
generate_protein(6)
```

```
[1] "HWTFMC"
```

Q4. Generate random protein sequences of length 6 to 13

> **Q4.** Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call generate_protein() eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp.

```
# Go through the sequence lengths from 6-13 amino acids
for(l in 6:13) {
# Print FASTA header line for each sequence
# '>id.l' is the label to the  sequence with its length
  cat(">id.", l, "\n", sep="")
# Generate a random protein sequence with length l and print the sequence
  cat ( generate_protein(l),"\n")
}
```

```
>id.6
VDNAET
>id.7
PLPFVKN
>id.8
EKWYQILD
>id.9
SVVWNCLIY
>id.10
GKAYQANCPC
>id.11
DYLLVNGMIKL
>id.12
KEGFTEIPVAIW
>id.13
HHCAKMIYTMESP
```

Are Our peptides "unique in nature"?

Q5. Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100	100	N
7	100	100	N
8	100	100	N
9	89	100	Y
10	89	90	Y
11	85	100	Y
12	100	75	Y
13	100	77	Y

Q5a. At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)?

My generated peptides start to look "unique in nature" at length 9.

Q5b. Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of

the sequence space (20^L for a peptide of length ^L) and to the size of the known protein universe.

Very short peptides are common in the nr database because the number of possible sequences (20^L) is relatively small. As ^L increases, the sequence space increases exponentially making random matches unlikely. Although the protein universe contains billions of amino acids across millions of proteins, it is still small compared to the vast number of possible longer peptide sequences.

Q6. In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

Based on my data, the minimum length is around 9 amino acids. This is reasonable because after the length of 9 amino acids, the possible sequence space becomes too large for most random sequences to already exist in nature. Additionally, presenting very short peptides would be a poor choice for the immune system since they are not unique and can appear in many proteins, including self-proteins. This increases the risk of failed recognition or triggering autoimmunity.